

xDSL Tutorial - part6

Milan Beric

September 10, 2026

1 Introduction

This tutorial is meant for developers and engineers who want to understand how modern ML compilers work under the hood, design their own IRs, and experiment with lowering, rewriting, and optimization passes in a practical, hands-on way.

We will use xDSL, a Python-based framework for building and experimenting with compiler intermediate representations (IRs) inspired by MLIR. It lets you define dialects, operations, types, and transformation passes directly in Python, without having to dive into C++ or the full LLVM/MLIR toolchain. This makes it ideal for rapid prototyping, teaching, and research in compiler design.

2 Prerequisites

Tutorial includes hands-on experience and practical exercises, having the following prerequisites will help you get the most out of it:

- **Python proficiency** – You should be comfortable writing and reading Python code, including classes, functions, and basic data structures.
- **Basic programming concepts** – Familiarity with concepts like control flow, data types, abstractions, and modular code design is expected.
- **ML and AI basics** – Understanding of machine learning and AI concepts (such as models, tensors, and operations) will help you follow compiler examples and motivations.

Basic terminology:

- **Dialect** – a namespace for operations and types (e.g. `arith`, `scf`, `tensor`, or your custom one).
- **Op (Operation)** – node in the IR graph: has operands, results, attributes, regions.
- **Type** – type of values (e.g. `tensor`, `integer`, `float`, etc.).
- **Module** – container of operations (top-level program).
- **Pass** – transformation or analysis that runs on modules/regions.
- **Pattern rewrite** – local rule: `match op(s) → rewrite to something else`.

3 Installation

If your Python version is ≥ 3.10 , you are good to go. Install fresh virtual environment as given:

```
1 mkdir xdsl-playground
2 cd xdsl-playground
3 python3 --version
4 python3 -m venv .venv
```

```
5 source .venv/bin/activate
```

Listing 1: Setting up environment

After activation, your shell prompt should show (.venv) in front. Now, install xdsl:

```
1 pip install xdsl
2 xdsl-opt --help
3 echo 'builtin.module {}' | xdsl-opt
```

Listing 2: xDSL installation

You should see a trivial IR printed back. Let's start with a very small hand-written IR file. Create `test.mlir`:

```
1 builtin.module {
2   func.func @add_two(%arg0 : i32, %arg1 : i32) -> i32 {
3     %0 = arith.addi %arg0, %arg1 : i32
4     return %0 : i32
5   }
6 }
```

Listing 3: xDSL example

and run:

```
1 xdsl-opt test.mlir
```

If xDSL has that dialect set enabled, it should parse and print it. The syntax is MLIR-like:

- `builtin.module` is the top module.
- `func.func` is a function.
- `arith.addi` is an arithmetic op.

Each op has:

- name (`arith.addi`)
- operands (`%arg0, %arg1`)
- result(s) (`%0`)
- type annotation (`: i32`)

4 Parsing mlir file

Now let's parse and print created mlir file from Python: Create a file `step1_parse_and_print.py`

```

1 from xdsl.context import Context
2 from xdsl.parser import Parser
3 from xdsl.dialects.builtin import Builtin
4 from xdsl.dialects.func import Func
5 from xdsl.dialects.arith import Arith
6
7 def main() -> None:
8     # Create context and load dialects you need
9     ctx = Context()
10    ctx.load_dialect(Builtin)
11    ctx.load_dialect(Func)
12    ctx.load_dialect(Arith)
13
14    # Read MLIR/xDSL file
15    with open("test.mlir", "r") as f:
16        text = f.read()
17
18    # Parse into a module
19    parser = Parser(ctx, text)
20    module = parser.parse_module()
21
22    # Print the module back out
23    print("=== Parsed module ===")
24    print(module)
25
26
27 if __name__ == "__main__":
28     main()

```

Listing 4: step1_parse_and_print.py

```

1 python3 step1_parse_and_print.py

```

Listing 5: Parse file

Exercises

- Create a mlir file for `mul_two` function (Hint: use `arith.muli`)
- Create a mlir file for `sub_two` function (Hint: use `arith.subi`)
- Create a mlir file for linear function `ax+b` (3 arguments)

5 Build a minimal module in Python

Now let's build a completely empty builtin.module in Python, just to touch the IR construction APIs. Create `step2_empty_module.py`:

```

1 from xdsl.context import Context
2 from xdsl.dialects.builtin import Builtin, ModuleOp
3
4 def main():
5     # Create context
6     ctx = Context()
7
8     # Load the builtin dialect into the context
9     ctx.load_dialect(Builtin)
10
11     # Create an empty module: builtin.module { }
12     module = ModuleOp(ops=[])
13
14     print("=== Empty module ===")
15     print(module)
16
17 if __name__ == "__main__":
18     main()

```

Listing 6: step2_empty_module.py

```

1 python3 step2_empty_module.py

```

Listing 7: Parse file

6 Create empty function

Goal in this section is to create a module that contains one function symbol (no real body yet), and print it. Create new file `step3_one_function.py` and add imports:

```

1 from xdsl.context import Context
2 from xdsl.ir import Region, Block
3 from xdsl.printer import Printer
4
5 from xdsl.dialects.builtin import Builtin, ModuleOp
6 from xdsl.dialects import func

```

Listing 8: Necessary imports

Create a new function below it, for building context.

```

1 def build_context() -> Context:
2     ctx = Context()
3     # Load the builtin dialect into the context
4     ctx.load_dialect(Builtin)
5     ctx.load_dialect(func.Func)
6     return ctx

```

Listing 9: Build context implementation

Create a new function in the same file, for building the module.

```
1 def build_module_with_empty_func(ctx: Context) -> ModuleOp:
2     # Create an empty module
3     module = ModuleOp(ops=[])
4
5     # Create an *empty* function: no arguments, no results (for now)
6     func_name = "my_empty_func"
7
8     # Most xdsl versions mirror MLIR's 'func.func' here:
9     #     func.FuncOp(name, (arg_types, result_types))
10    # With no args/results: ([], [])
11    fn = func.FuncOp(func_name, ([], []))
12
13    # Make the function a declaration by giving it an empty body region.
14    fn.body = Region()
15
16    # Replace the whole body region
17    # with a Region that has a single Block containing 'fn'.
18    module.body = Region(Block(ops=[fn]))
19
20    return module
```

Listing 10: Build module implementation

Add main function and invoke module creation:

```
1 def main() -> None:
2     ctx = build_context()
3     module = build_module_with_empty_func(ctx)
4
5     # Create a printer
6     printer = Printer()
7     printer.print_op(module)
8     print()
9
10 if __name__ == "__main__":
11     main()
```

Listing 11: Main function implementation

Run the script from the terminal:

```
1 python3 step3_one_function.py
```

You should get something like:

```
1 builtin.module {
2     func.func @my_empty_func() -> ()
3 }
```

Listing 12: Console print

7 Add operations inside the function

Create a new file `step4_func_with_body.py` and import packages:

```
1 from xdsl.context import Context
2 from xdsl.ir import Region, Block
3 from xdsl.dialects.builtin import Builtin, ModuleOp, i32
4 from xdsl.dialects import func, arith
```

Listing 13: Adding imports

Build context once again and provide main function in the same way as in previous step.
Build module after that:

```
1 def build_module_with_body(ctx: Context) -> ModuleOp:
2     # Create constants: 1 and 2 as i32
3     c1 = arith.ConstantOp.from_int_and_width(1, 32)
4     c2 = arith.ConstantOp.from_int_and_width(2, 32)
5
6     # Add them: %2 = arith.addi %0, %1 : i32
7     add = arith.AddiOp(c1.result, c2.result)
8
9     # Return the result
10    ret = func.ReturnOp(add.result)
11
12    # Build a block that contains all operations in order
13    body_block = Block(ops=[c1, c2, add, ret])
14
15    # Create a function @my_func() -> i32 with that body
16    fn = func.FuncOp(
17        "my_func",
18        ([], [i32]), # no arguments, one i32 result
19    )
20    fn.body = Region(body_block)
21
22    # Put the function inside a module
23    module = ModuleOp(ops=[fn])
24    return module
```

Listing 14: Build module

Resulting IR should look like:

```
1 builtin.module {
2   func.func @my_func() -> i32 {
3     %0 = arith.constant 1 : i32
4     %1 = arith.constant 2 : i32
5     %2 = arith.addi %0, %1 : i32
6     func.return %2 : i32
7   }
```

```
8 }
```

Listing 15: Printed module

At this point, we have:

- a working context
- a working module
- a working func.func
- a working body block with real operations in xDSL

8 Creating new dialect

Add imports and define custom dialect.

```
1 from xdsl.context import Context
2 from xdsl.ir import Dialect
3 from xdsl.dialects.builtin import Builtin
4 from xdsl.dialects import func
5
6
7 # For now: Empty HiCompiler dialect (no ops / attributes yet)
8 HiCompiler = Dialect(
9     "hc",          # dialect name: will give ops like 'hc.foo' later
10    (),            # operations (we will add ops here in later steps)
11    (),            # attributes / types (also later)
12 )
```

Listing 16: Define new dialect

The point of this step is just is just to see how can we register and load the dialect.

```
1 def main() -> None:
2     ctx = Context()
3
4     # Load the built-in dialects
5     ctx.load_dialect(Builtin)
6     ctx.load_dialect(func.Func)
7
8     # Now load your custom HiCompiler dialect instance
9     ctx.load_dialect(HiCompiler)
10
11     # The point is just: 'can we register and load the dialect without
12     # errors?'
13     print(ctx)
14     print("\nHiCompiler dialect registered and loaded successfully.")
```

```

15 if __name__ == "__main__":
16     main()

```

Listing 17: Define new dialect

9 Define first operation

For easier maintenance, we will split dialect into a separate file and extend it with new operation. Create `hc_dialect.py` file:

```

1 from xdsl.ir import Dialect
2 from xdsl.irdl import (
3     IRDLOperation,
4     irdl_op_definition,
5     operand_def,
6     result_def,
7 )
8 from xdsl.dialects.builtin import IntegerType
9
10 # -----
11 # Operation: hc.add
12 # -----
13 @irdl_op_definition
14 class HCAdd(IRDLOperation):
15     name = "hc.add"
16
17     # Two i32 operands
18     lhs = operand_def(IntegerType)
19     rhs = operand_def(IntegerType)
20
21     # One i32 result
22     res = result_def(IntegerType)
23
24 # -----
25 # Dialect: HiCompiler
26 # -----
27 HiCompiler = Dialect(
28     "hc",
29     (HCAdd,),      # register operations here
30     (),           # attrs
31 )

```

Listing 18: Define and register add operation

Use newly created operation in the module:

```

1 from xdsl.context import Context
2 from xdsl.ir import Region, Block

```

```

3 from xdsl.dialects.builtin import Builtin, ModuleOp, i32
4 from xdsl.dialects import func, arith
5
6 from hc_dialect import HiCompiler, HCAdd
7
8
9 def build_context() -> Context:
10     ctx = Context()
11     ctx.load_dialect(Builtin)
12     ctx.load_dialect(func.Func)
13     ctx.load_dialect(arith.Arith)
14     ctx.load_dialect(HiCompiler)    # <-- our custom dialect
15     return ctx
16
17
18 def build_module(ctx: Context) -> ModuleOp:
19     # Constants: 1 and 2
20     c1 = arith.ConstantOp.from_int_and_width(1, 32)
21     c2 = arith.ConstantOp.from_int_and_width(2, 32)
22
23     # Our custom operation: %2 = hc.add %0, %1 : i32
24     hc_add = HCAdd(
25         operands=[c1.result, c2.result],
26         result_types=[i32],
27     )
28
29     # Return the result
30     ret = func.ReturnOp(hc_add.results[0])
31
32     # Block containing the constants + hc_add + return
33     block = Block(ops=[c1, c2, hc_add, ret])
34
35     # Function returning i32
36     fn = func.FuncOp("my_func", ([], [i32]))
37     fn.body = Region(block)
38
39     # Module
40     module = ModuleOp(ops=[fn])
41     return module
42
43
44 def main():
45     ctx = build_context()
46     module = build_module(ctx)
47     print(module)
48
49

```

```

50 if __name__ == "__main__":
51     main()

```

Listing 19: Build module with add operation

You should see something like:

```

1 builtin.module {
2   func.func @my_func() -> i32 {
3     %0 = arith.constant 1 : i32
4     %1 = arith.constant 2 : i32
5     %2 = "hc.add"(%0, %1) : (i32, i32) -> i32
6     func.return %2 : i32
7   }
8 }

```

Listing 20: IR with hc.add operation

We have a fully functioning custom MLIR dialect and operation in xDSL.

10 Lowering hc.add → arith.addi

In this section we will define a rewrite pattern that will match hc.add operation and replace it with arith.addi. This will be the first step in our implementation for lowering the code. Modify import section by adding inspect and Operation packages.

```

1 import inspect
2 from xdsl.ir import Region, Block, Operation
3
4 from xdsl.pattern_rewriter import (
5     RewritePattern,
6     PatternRewriter,
7     GreedyRewritePatternApplier,
8     PatternRewriteWalker,
9 )

```

Listing 21: New imports

Create lowering pattern:

```

1 # -----
2 #   Lowering pattern: hc.add -> arith.addi
3 # -----
4
5 class LowerHCAddPattern(RewritePattern):
6     def match_and_rewrite(self, op: Operation, rewriter: PatternRewriter
7         ):
8         # Debug: uncomment if you want to see traversal
9         # print("VISIT:", op.name)

```

```

10     if op.name != "hc.add":
11         return
12
13     # hc.add has two operands
14     lhs, rhs = op.operands
15
16     # Replacement op
17     new_add = arith.AddiOp(lhs, rhs)
18
19     rewriter.replace_op(
20         op,
21         new_ops=[new_add],
22         new_results=[new_add.result],
23         safe_erase=True,
24     )

```

Listing 22: Lowering pattern for add operation

Apply lowering using newly written pattern.

```

1 def apply_lowering(module: ModuleOp) -> None:
2     applier = GreedyRewritePatternApplier([LowerHCAddPattern()])
3
4     # Some xdsl builds have extra kwargs; enable recursion if available.
5     init_sig = inspect.signature(PatternRewriteWalker.__init__)
6     kwargs = {}
7     for k in ("walk_regions", "apply_recursively", "walk_into_regions"):
8         if k in init_sig.parameters:
9             kwargs[k] = True
10
11     walker = PatternRewriteWalker(applier, **kwargs)
12
13     # Walk/rewrite the function bodies explicitly
14     for top_block in module.body.blocks:
15         for op in top_block.ops:
16             if isinstance(op, func.FuncOp):
17                 if hasattr(walker, "rewrite_region"):
18                     walker.rewrite_region(op.body)
19                 else:
20                     # fallback: rewrite_op on the function op if that
21                     # implies walking regions in your build
22                     walker.rewrite_op(op)

```

Listing 23: Applying lowering

Redefine main function so that it prints module before and after lowering.

```

1 def main():
2     ctx = build_context()
3     module = build_module(ctx)

```

```

4
5     print("=== BEFORE LOWERING ===")
6     print(module)
7     apply_lowering(module)
8     print("\n=== AFTER LOWERING ===")
9     print(module)

```

Listing 24: Main function

In the logs, you should see that operations were really lowered.

Exercises

- Extend dialect with `mul` operation and implement lowering
- Build module with `mul` operation
- Extend dialect with `sub` operation and implement lowering
- Build module with `sub` operation

11 Add more operations

As a next step, we will add `mul` and `sub` operations and corresponding lowering rules. To do that, first extend dialect:

```

1 @irdl_op_definition
2 class HCMul(IRDLOperation):
3     name = "hc.mul"
4     lhs = operand_def(IntegerType)
5     rhs = operand_def(IntegerType)
6     res = result_def(IntegerType)
7
8 @irdl_op_definition
9 class HCSub(IRDLOperation):
10     name = "hc.sub"
11     lhs = operand_def(IntegerType)
12     rhs = operand_def(IntegerType)
13     res = result_def(IntegerType)

```

Listing 25: Adding new operations in `hc_dialect.py`

Also, register new operations:

```

1 HiCompiler = Dialect(
2     "hc",
3     (HCAdd, HCMul, HCSub),      # register operations here
4     (), # attrs
5 )

```

Listing 26: Registering new operations in `hc_dialect.py`

In the main file, import operations in order to use them:

```
1 from hc_dialect import HiCompiler, HCAdd, HCSub, HCMul
```

Listing 27: Importing operations

Modify `build_module` function, so that it contains `HCMul` and `HCSub`.

```
1 c1 = arith.ConstantOp.from_int_and_width(1, 32)
2 c2 = arith.ConstantOp.from_int_and_width(2, 32)
3 c3 = arith.ConstantOp.from_int_and_width(3, 32)
4 c4 = arith.ConstantOp.from_int_and_width(4, 32)
5
6 hc_add = HCAdd(
7     operands=[c1.result, c2.result],
8     result_types=[i32],
9 )
10 hc_mul = HCMul(
11     operands=[hc_add.results[0], c3.result],
12     result_types=[i32],
13 )
14 hc_sub = HCSub(
15     operands=[hc_mul.results[0], c4.result],
16     result_types=[i32],
17 )
18
19 # Return the result
20 ret = func.ReturnOp(hc_sub.results[0])
21
22 # Block containing the constants + hc ops + return
23 block = Block(ops=[c1, c2, c3, c4, hc_add, hc_mul, hc_sub, ret])
24 ...
```

Listing 28: build context snippet

Lastly, modify `match_and_rewrite` function:

```
1 def match_and_rewrite(self, op: Operation, rewriter: PatternRewriter):
2     # Debug: uncomment if you want to see traversal
3     # print("VISIT:", op.name)
4
5     if op.name not in ("hc.add", "hc.mul", "hc.sub"):
6         return
7
8     # operations have two operands
9     lhs, rhs = op.operands
10    if op.name == "hc.add":
11        new_op = arith.AddiOp(lhs, rhs)
12    elif op.name == "hc.mul":
13        new_op = arith.MuliOp(lhs, rhs)
```

```

14     elif op.name == "hc.sub":
15         new_op = arith.SubiOp(lhs, rhs)
16     else:
17         raise Exception("Operation not supported")
18
19     rewriter.replace_op(
20         op,
21         new_ops=[new_op],
22         new_results=[new_op.result],
23         safe_erase=True,
24     )

```

Listing 29: build_context snippet

apply_lowering and main functions remain the same. Run the script and you should see `hc.add`, `hc.mul` and `hc.sub` lowered to `arith.addi`, `arith.muli` and `arith.subi` respectively.

Our `hc_dialect` is a front-end/high level IR. In the following example:

```

1 %2 = hc.add %0, %1 : i32

```

Listing 30: HC instruction example

- We define how to add two values in our language.
- Semantics are defined by us, not by the compiler infrastructure.
- Compiler doesn't know how to optimize it or emit code for it.

After lowering `hc.add` → `arith.addi` we have:

- Precise integer semantics
- Defined overflow behavior
- Standardized typing rules
- Access to existing optimizations

```

1 hc.*          (our DSL)
2   ↓
3 arith.*       (standard math)
4   ↓
5 scf.*         (loops, control flow)
6   ↓
7 llvm.*        (machine model)
8   ↓
9 x86 / ARM     (real code)

```

Listing 31: Typical pipeline

12 Adding ReLu operator

As before, extend `hc_dialect.py` with `HCRelu`. Note: This time we only have one operand. Register operation. Here is one helper function that we will use:

```
1 def _const_i32(value: int) -> arith.ConstantOp:  
2     return arith.ConstantOp.from_int_and_width(value, 32)
```

Listing 32: Const conversion helper

Change `match_and_rewrite` function from the main file:

```
1 def match_and_rewrite(self, op: Operation, rewriter: PatternRewriter):  
2     ...  
3     # ----- hc.relu -----  
4     if op.name == "hc.relu":  
5         x = op.operands[0]  
6         c0 = _const_i32(0)  
7  
8         if hasattr(arith, "MaxSIOp"):  
9             maxop = arith.MaxSIOp(x, c0.result)  
10            rewriter.replace_op(  
11                op,  
12                new_ops=[c0, maxop],  
13                new_results=[maxop.result],  
14                safe_erase=True,  
15            )  
16            return  
17        else:  
18            raise RuntimeError(  
19                "Cannot lower hc.relu: arith.MaxSIOp needed."  
20            )
```

Listing 33: Add logic for HCRelu lowering

Change `build_module` function to include `HCRelu` operation and run the script. If `arith.maxsi(x,0)` is available, you should see `hc.relu` lowered to 2 operations: `arith.constant` and `arith.maxsi`.

13 Constant folding

Now that everything is `arith`, we can start doing real optimization passes:

- Fold `arith.addi/muli/subi` when both operands are constants
- Simplify `relu(constant)` to a constant
- Remove dead constants no longer used

What we want to do as a result is a pass that turns `relu((1+2)*3-10)` into a single `arith.constant 0 : i32` (or the computed value if positive).

Change import section:

```
1 from xdsl.ir import Region, Block, Operation, SSAValue
```

Listing 34: Importing SSAValue

Delete `apply_lowering` function and replace it with:

```
1 def apply_pass(module: ModuleOp, pattern: RewritePattern) -> None:
2     applier = GreedyRewritePatternApplier([pattern()])
3
4     # Some xdsl builds have extra kwargs; enable recursion if available.
5     init_sig = inspect.signature(PatternRewriteWalker.__init__)
6     kwargs = {}
7     for k in ("walk_regions", "apply_recursively", "walk_into_regions"):
8         if k in init_sig.parameters:
9             kwargs[k] = True
10
11     walker = PatternRewriteWalker(applier, **kwargs)
12
13     # Walk/rewrite the function bodies explicitly
14     for top_block in module.body.blocks:
15         for op in top_block.ops:
16             if isinstance(op, func.FuncOp):
17                 if hasattr(walker, "rewrite_region"):
18                     walker.rewrite_region(op.body)
19                 elif hasattr(walker, "rewrite_op"):
20                     walker.rewrite_op(op)
21             else:
22                 raise RuntimeError("No suitable rewrite entrypoint
                                     found on PatternRewriteWalker.")
```

Listing 35: Apply pass implementation

since we are going to reuse it for constant folding.

Add a couple of helper functions to deal with `IntAttr` and extract underlying integer:

```
1 # -----
2 # Helper functions
3 # -----
4
5 def _defining_op(val: SSAValue) -> Operation | None:
6     # SSAValue in xdsl typically has .owner (def op)
7     return getattr(val, "owner", None)
8
9 def _get_const_int_binop_args(op: Operation):
10     """Return (a, b) if both operands are integer constants, else None."""
```

```

11     if len(op.operands) != 2:
12         return None
13
14     lhs, rhs = op.operands
15     lhs_op = _defining_op(lhs)
16     rhs_op = _defining_op(rhs)
17     if lhs_op is None or rhs_op is None:
18         return None
19
20     a = _int_from_constant_op(lhs_op)
21     b = _int_from_constant_op(rhs_op)
22     if a is None or b is None:
23         return None
24
25     return a, b
26
27 def _int_from_constant_op(op: Operation) -> int | None:
28     """
29     Extract integer from arith.constant.
30     Returns None if not an integer constant we can read.
31     """
32     if op.name != "arith.constant":
33         return None
34
35     # In many versions: arith.ConstantOp has attribute '.value'
36     v = getattr(op, "value", None)
37     if v is None:
38         return None
39
40     # Common shapes: IntegerAttr(value=...), or has '.value'/'data'
41     for attr_name in ("value", "data"):
42         if hasattr(v, attr_name):
43             try:
44                 return int(getattr(v, attr_name))
45             except Exception:
46                 pass
47
48     # Last resort: try to parse int from string like "1 : i32"
49     # (kept minimal; only if the above fails)
50     try:
51         s = str(op)
52         # crude parse: find "arith.constant " then grab next token
53         if "arith.constant" in s:
54             tok = s.split("arith.constant", 1)[1].strip().split()[0]
55             return int(tok)
56     except Exception:
57         pass

```

```

58
59     return None
60
61
62 def _result_width(op: Operation, default: int = 32) -> int:
63     # Try to read integer width from result type if available
64     if not op.results:
65         return default
66
67     ty = op.results[0].type
68     w = getattr(ty, "width", None)
69     if w is None:
70         return default
71
72     # In our xdsl, width is an IntAttr, not a Python int
73     for attr_name in ("data", "value"):
74         if hasattr(w, attr_name):
75             return int(getattr(w, attr_name))
76
77     # fallback: try direct int conversion if supported
78     try:
79         return int(w)
80     except Exception:
81         return default

```

Listing 36: Helper function

```

1  # -----
2  # Constant folding pattern
3  # -----
4  class FoldArithInts(RewritePattern):
5      def match_and_rewrite(self, op: Operation, rewriter: PatternRewriter
6          ):
7          # Special-case: fold relu lowering if it became maxsi
8          if op.name == "arith.maxsi":
9              args = _get_const_int_binop_args(op)
10             if args is None:
11                 return
12             a, b = args
13             width = _result_width(op, 32)
14             c = arith.ConstantOp.from_int_and_width(max(a, b), width)
15             rewriter.replace_op(op, new_ops=[c], new_results=[c.result],
16                 safe_erase=True)
17             return
18
19     # Fold binary integer ops when both operands are integer
20     constants
21     if op.name not in ("arith.addi", "arith.muli", "arith.subi"):

```

```

19         return
20
21     args = _get_const_int_binop_args(op)
22     if args is None:
23         return
24     a, b = args
25
26     res_by_op = {
27         "arith.addi": a + b,
28         "arith.muli": a * b,
29         "arith.subi": a - b,
30     }
31     res = res_by_op[op.name]
32
33     width = _result_width(op, 32)
34     c = arith.ConstantOp.from_int_and_width(res, width)
35     rewriter.replace_op(op, new_ops=[c], new_results=[c.result],
        safe_erase=True)

```

Listing 37: Constant folding pattern implementation

Rename LowerHCAddPattern class to LowerHCPattern, since it now contains more operations.
Add constant folding pass in the main function:

```

1 def main():
2     ctx = build_context()
3     module = build_module(ctx)
4
5     print("=== HIGH LEVEL (hc.*) ===")
6     print(module)
7
8     # Apply lowering
9     apply_pass(module, LowerHCPattern)
10
11     print("\n=== AFTER LOWERING (arith.*) ===")
12     print(module)
13
14     # Apply constant folding
15     apply_pass(module, FoldArithInts)
16     print("\n=== AFTER CONSTANT FOLDING ===")
17     print(module)

```

Listing 38: Main function

14 Dead code elimination

Next, will add a tiny DCE pass that removes ops whose results have no use. Focus here is to erase ops that:

- are not terminators (e.g., not `func.return`)
- have results
- all results have zero uses
- `op` is in a “safe-to-drop” set

We will run this in a loop until no more ops can be removed. In most xDSL versions, the safe sequence is:

- `op.detach()` (removes it from its block / unlinks)
- then `op.erase()` (frees it, asserts parent is `None`)

First, we will create some helper functions. Start with the function that returns how many times an SSA value is used in the IR.

```

1 # -----
2 # Dead code elimination
3 # -----
4 def _num_uses(v: SSAValue) -> int:
5     uses = getattr(v, "uses", None)
6     if uses is None:
7         return 0
8     try:
9         return len(uses)
10    except TypeError:
11        # Sometimes uses is an iterator/generator-like
12        return sum(1 for _ in uses)

```

Listing 39: Counting uses

In xDSL (like in MLIR), every `SSAValue` keeps track of its uses — i.e. which operations consume that value as an operand. Next, check conditions for the simple dce case:

```

1 def _is_trivially_dceable(op: Operation) -> bool:
2     """
3     Conservative: only remove arith ops (constants and arithmetic) that
4     have
5     no users. Not covering memref/loads/stores/calls etc. for now.
6     """
7     if op.name == "func.return":
8         return False
9
10    # Only drop ops from these dialects for now
11    if op.name.startswith("arith."):
12        return True
13
14    # If we also want to also drop leftover hc.* after lowering, we
15    could include:

```

```

14     # if op.name.startswith("hc."):
15     #     return True
16
17     return False

```

Listing 40: Checking if it is trivial to do dce

Implement a function that performs one pass of Dead Code Elimination (DCE) over a single Block in xDSL:

```

1 def dce_block(block: Block) -> bool:
2     """
3     One pass of DCE over a block.
4     Returns True if anything was erased.
5     """
6     changed = False
7
8     # Iterate backwards: safer when erasing
9     for op in list(block.ops)[::-1]:
10        # Must be safe to erase
11        if not _is_trivially_dceable(op):
12            continue
13
14        # If op has no results, keep it (could be side-effecting;
15        # conservative)
16        if len(op.results) == 0:
17            continue
18
19        # If any result is used, keep it
20        if any(_num_uses(res) > 0 for res in op.results):
21            continue
22
23        # Otherwise erase
24        _erase_op(op)
25        changed = True
26
27    return changed

```

Listing 41: One DCE pass implementation

Notice that by iterating backwards, we eliminate leaf nodes first.

Implement a function that runs Dead Code Elimination (DCE) repeatedly over all function bodies in a module until nothing more can be removed.

```

1 def apply_dce(module) -> None:
2     """
3     Repeatedly run DCE on all function body blocks until fixpoint.
4     """
5     changed = True
6     while changed:

```

```

7         changed = False
8         for top_block in module.body.blocks:
9             for op in list(top_block.ops):
10                 if isinstance(op, func.FuncOp):
11                     for body_block in op.body.blocks:
12                         if dce_block(body_block):
13                             changed = True

```

Listing 42: Applying DCE

Let's break it down:

- It walks through all functions in the module
- Runs `dce_block()` on every block inside each function
- If anything was removed → run everything again
- Stops only when no more dead code can be eliminated

Lastly, implement a helper function that first removes operation from its parent block, and then erases it.

```

1 def _erase_op(op: Operation) -> None:
2     # Detach from parent block first (required in our xdsl)
3     if hasattr(op, "detach"):
4         op.detach()
5     else:
6         raise RuntimeError("Operation.detach() not found; cannot safely
7                               DCE ops.")
8     op.erase()

```

Listing 43: Erasing operation

Now, use it in the main function:

```

1     ...
2     apply_dce(module)
3     print("\n=== AFTER DCE ===")
4     print(module)

```

Listing 44: Adding DCE in the main body

In the DCE section, you should only see one constants, others have been removed:

```

1 === AFTER DCE ===
2 builtin.module {
3     func.func @my_func() -> i32 {
4         %0 = arith.constant 0 : i32
5         func.return %0 : i32
6     }
7 }

```

Listing 45: DCE output

15 Analysis

Purpose of analysis is to compute information about the IR without changing it, that can drive later optimizations/codegen. We will implement two classic analyses on a function body block:

- Use-def report - for each SSA result, list which ops use it
- Block-local liveness - compute the live set after each op in a straight-line block

For typing annotation, import additional packages:

```
1 from typing import Any, Iterable
```

Listing 46: Import typing

Create a couple of helper function:

```
1 # -----
2 # Analysis
3 # -----
4
5 # Helper functions
6 def _iter_uses(val: SSAValue) -> Iterable[Any]:
7     uses = getattr(val, "uses", None)
8     if uses is None:
9         return []
10    return uses
11
12 def _use_owner_op(use: Any) -> Operation | None:
13     """
14     Try to get the user operation from a Use object.
15     """
16     for attr in ("operation", "op", "owner"):
17         u = getattr(use, attr, None)
18         if isinstance(u, Operation):
19             return u
20     # some Use objects store the operation under .operation on a field
21     return None
22
23 def _fmt_val(v: SSAValue) -> str:
24     try:
25         return str(v) # often prints as %0
26     except Exception:
27         return f"<ssa@{id(v):x}>"
28
29 def _fmt_op(op: Operation) -> str:
30     # compact one-liner label
31     return f"{op.name}"
32
33 def _val_key(v: SSAValue) -> int:
```

```

34     """
35     Use id() as a stable key even if SSAValue is not hashable in some
        builds.
36     """
37     return id(v)
38
39 def _collect_operands(op: Operation) -> list[SSAValue]:
40     return list(getattr(op, "operands", []))
41
42 def _collect_results(op: Operation) -> list[SSAValue]:
43     return list(getattr(op, "results", []))

```

Listing 47: Analysis helper functions

- `_iter_uses` — Every `SSAValue` knows which operations use it, and that information is stored in `val.uses`. This function returns an iterable, even if `.uses` does not exist.
- `_use_owner_op` — Extracts the operation that uses a given `SSAValue` in a defensive way that tolerates different `Use` representations.
- `_fmt_val` — Creates a safe, human-readable string representation of an `SSAValue`.
- `_fmt_op` — Provides a compact textual label for an operation.
- `_val_key` — Creates a stable, hashable identity key for an `SSAValue`, even if the `SSAValue` itself cannot be used as a dictionary key.
- `_collect_operands` — Collects an operation's operands as a plain Python list.
- `_collect_results` — Collects an operation's results as a concrete Python list.

Create a function for printing use-def report:

```

1  # -----
2  # Analysis 1: Use-def report
3  # -----
4  def print_use_def(block: Block) -> None:
5      print("\n--- Use-def report (per SSA result) ---")
6      for op in block.ops:
7          for res in _collect_results(op):
8              users: list[str] = []
9              for use in _iter_uses(res):
10                 uop = _use_owner_op(use)
11                 if uop is None:
12                     users.append("<unknown-user>")
13                 else:
14                     users.append(_fmt_op(uop))
15             users_s = ", ".join(users) if users else "(no uses)"

```

```

16         print(f"{_fmt_val(res)} defined by {_fmt_op(op)} -> used by
           : {users_s}")

```

Listing 48: Creating use-def report

Create a function for printing liveness report:

```

1  # -----
2  # Analysis 2: Block-local liveness (straight-line)
3  # -----
4  def compute_liveness(block: Block):
5      """
6      Returns a list 'live_after[i]': dict of {id(ssa): ssa} live AFTER
7      block.ops[i].
8      """
9      ops = list(block.ops)
10     live_after: list[dict[int, SSAValue]] = [dict() for _ in ops]
11
12     live: dict[int, SSAValue] = {}
13
14     # Seed with values used by the terminator (usually func.return)
15     if ops:
16         term = ops[-1]
17         for v in _collect_operands(term):
18             live[_val_key(v)] = v
19
20     # Walk backwards
21     for i in range(len(ops) - 1, -1, -1):
22         op = ops[i]
23
24         # live_after for this op = current live (after processing later
25         # ops)
26         live_after[i] = dict(live)
27
28         # Remove defs (results) from live (killing)
29         for res in _collect_results(op):
30             live.pop(_val_key(res), None)
31
32         # Add uses (operands) to live
33         for v in _collect_operands(op):
34             live[_val_key(v)] = v
35
36     return live_after

```

Listing 49: Creating block-local liveness report

Implement a function that will print liveness based on previously added `compute_liveness`.

```

1 def print_liveness(block: Block) -> None:
2     live_after = compute_liveness(block)

```

```

3
4     print("\n--- Block liveness (live set AFTER each op) ---")
5     ops = list(block.ops)
6     for i, op in enumerate(ops):
7         live_vals = sorted((_fmt_val(v) for v in live_after[i].values())
8                             )
9         live_s = ", ".join(live_vals) if live_vals else "(empty)"
10        print(f"[{i:02d}] after {_fmt_op(op):>12} : {live_s}")

```

Listing 50: Creating a function for printing liveness

Implement a function for printing report:

```

1 def analyze(module: ModuleOp):
2     # Analyze each function body block
3     for top_block in module.body.blocks:
4         for op in top_block.ops:
5             if isinstance(op, func.FuncOp):
6                 print(f"\n=====")
7                 print(f"Analyzing Function: {op.sym_name.data if hasattr(
8                     op, 'sym_name') else 'func'}")
9                 print(f"=====")
10
11                for body_block in op.body.blocks:
12                    print_use_def(body_block)
13                    print_liveness(body_block)

```

Listing 51: Function body block analysis

and call it in the main function, after lowering and optimizations:

```

1 ...
2 analyze(module)

```

Listing 52: `analyze` call in the main

Disable constant folding and dead code elimination and run script again. Observe the results.

16 Refactoring time

So far, we have placed all of our logic inside the main function to keep things simple and focused on understanding the core concepts. Now, as the codebase grows, we will start refactoring it into smaller, well-structured components to improve readability, modularity, and maintainability. What we are trying to achieve is following structure:

```

1 middle_end
2 |-- analysis.py
3 |-- constant_folding.py
4 |-- dead_code_elimination.py
5 |-- hc_lowering.py

```

```

6 |-- __init__.py
7 |-- pipeline.py
8 hc_dialect.py
9 hc_main.py

```

Listing 53: Folder structure

- Create `middle_end` folder and given Python files
- Create an empty `__init__.py`
- Extract sections from `hc_main.py` into appropriate files

`pipeline.py` serves as a central integration point that orchestrates and connects multiple compiler passes into a coherent transformation pipeline. Inside of it, add imports:

```

1 import inspect
2 from dataclasses import dataclass
3 from xdsl.dialects import func
4 from xdsl.dialects.builtin import ModuleOp
5 from xdsl.pattern_rewriter import (
6     GreedyRewritePatternApplier,
7     PatternRewriteWalker,
8     RewritePattern,
9 )
10
11 from .analysis import analyze
12 from .constant_folding import FoldArithInts
13 from .dead_code_elimination import apply_dce
14 from .hc_lowering import LowerHCPattern

```

Listing 54: Pipeline imports

Bellow it, create configuration class that will help us to toggle specific passes easier.

```

1 # -----
2 # Configuration object
3 # -----
4 @dataclass(frozen=True)
5 class MiddleEndPipelineConfig:
6     run_analysis: bool = False
7     debug_mode: bool = False
8
9     apply_lowering: bool = True
10    apply_constant_folding: bool = True
11    apply_dce: bool = True

```

Listing 55: Configuration class

One helper function, based on previous implementation:

```

1 def _apply_pass(module: ModuleOp, pattern: RewritePattern) -> None:
2     applier = GreedyRewritePatternApplier([pattern()])
3
4     # Some xdsl builds have extra kwargs; enable recursion if available.
5     init_sig = inspect.signature(PatternRewriteWalker.__init__)
6     kwargs = {}
7     for k in ("walk_regions", "apply_recursively", "walk_into_regions"):
8         if k in init_sig.parameters:
9             kwargs[k] = True
10
11     walker = PatternRewriteWalker(applier, **kwargs)
12
13     # Walk/rewrite the function bodies explicitly
14     for top_block in module.body.blocks:
15         for op in top_block.ops:
16             if isinstance(op, func.FuncOp):
17                 if hasattr(walker, "rewrite_region"):
18                     walker.rewrite_region(op.body)
19                 elif hasattr(walker, "rewrite_op"):
20                     walker.rewrite_op(op)
21             else:
22                 raise RuntimeError("No suitable rewrite entrypoint
                                     found on PatternRewriteWalker.")

```

Listing 56: apply_pass helper function

And the middle end pipeline itself:

```

1 # -----
2 # Pipeline
3 # -----
4 @dataclass
5 class MiddleEndPipeline:
6     def __init__(self, config: MiddleEndPipelineConfig):
7         self.config = config
8
9     def apply_passes(self, module: ModuleOp) -> None:
10         if self.config.apply_lowering:
11             # Apply lowering
12             _apply_pass(module, LowerHCPattern)
13             self._print_module(module, "=== AFTER LOWERING (arith.*)
                                     ===")
14
15         if self.config.apply_constant_folding:
16             # Apply constant folding
17             _apply_pass(module, FoldArithInts)
18             self._print_module(module, "=== AFTER CONSTANT FOLDING ===")
19

```

```

20
21     if self.config.apply_dce:
22         # Apply dead code elimination
23         apply_dce(module)
24         self._print_module(module, "=== AFTER DEAD CODE ELIMINATION
25         ===")
26
27     if self.config.run_analysis:
28         # Analyze pass
29         analyze(module)
30
31     def _print_module(self, module: ModuleOp, message: str):
32         if self.config.debug_mode:
33             print(message + "\n")
34             print(module)

```

Listing 57: Middle end pipeline class

For our main file, provide necessary imports:

```

1 from middle_end.pipeline import (
2     MiddleEndPipeline, MiddleEndPipelineConfig
3 )

```

Listing 58: Middle-end imports

Then, create wanted configuration and apply middle-end passes:

```

1 def main():
2     ctx = build_context()
3     module = build_module(ctx)
4
5     print("=== HIGH LEVEL (hc.*) ===")
6     print(module)
7
8
9     config = MiddleEndPipelineConfig(
10         apply_lowering=True,
11         apply_constant_folding=True,
12         apply_dce=True,
13         run_analysis=False,
14         debug_mode=True,
15     )
16
17     middle_end_pipeline = MiddleEndPipeline(config)
18     middle_end_pipeline.apply_passes(module)

```

Listing 59: Middle-end imports

MiddleEndPipeline now encapsulates the execution of middle-end compiler passes and drives them according to a configurable setup. Its `apply_passes` method conditionally applies lower-

ing, constant folding, dead code elimination, and analysis based on flags defined in `MiddleEndPipelineConfig`, while `_print_module` provides optional debug output after each stage to help trace how the IR evolves through the pipeline.

17 Simulator

A simulator is a component that models the behavior of a system by executing its operations, sometimes including hardware-level details such as memory, timing, or architectural constraints. In this tutorial, we will use an interpreter, which directly executes our IR operations to validate correctness without modeling low-level hardware behavior.

Create 'simulator' folder next to your `hc_main.py` and `hc_dialect.py`:

```
1 simulator
2 |-- __init__.py
3 |-- interpreter.py
```

Listing 60: Simulator folder structure

First add helper functions:

```
1 from typing import Any
2 from xdsl.ir import Operation, SSAValue
3 from xdsl.dialects import func
4
5 # -----
6 # Helpers to read constants
7 # -----
8
9 def _int_from_attr(attr: Any) -> int | None:
10     """
11     Extract an int from common xdsl attribute shapes (IntegerAttr/
12     IntAttr, etc.).
13     """
14     if attr is None:
15         return None
16
17     # Often: attr.value is IntAttr; or attr.data is int; or nested.
18     for k in ("data", "value"):
19         if hasattr(attr, k):
20             inner = getattr(attr, k)
21             # inner might again be an attr
22             for k2 in ("data", "value"):
23                 if hasattr(inner, k2):
24                     try:
25                         return int(getattr(inner, k2))
26                     except Exception:
27                         pass
28     try:
```

```

28         return int(inner)
29     except Exception:
30         pass
31
32     try:
33         return int(attr)
34     except Exception:
35         return None
36
37
38 def _const_value(op: Operation) -> int | None:
39     """
40     arith.constant -> python int
41     """
42     if op.name != "arith.constant":
43         return None
44     v = getattr(op, "value", None)
45     return _int_from_attr(v)

```

Listing 61: Helper functions

Create Interpreter class that will let us instantiate our interpreter:

```

1  # -----
2  # Interpreter
3  # -----
4
5  class Interpreter:
6      def __init__(self):
7          # SSAValue is not always hashable across versions -> use id()
8          self.env: dict[int, int] = {}
9
10     def _get(self, v: SSAValue) -> int:
11         return self.env[id(v)]
12
13     def _set(self, v: SSAValue, value: int) -> None:
14         print(f"Set SSAValue: {v} = {int(value)}")
15         self.env[id(v)] = int(value)

```

Listing 62: Interpreter class

Implement `run_block` method inside of this class. It deals with both, `arith` and `hc` instructions.

```

1  def run_block(self, block) -> int:
2      """
3      Execute a single basic block (straight-line).
4      Returns the integer from func.return.
5      """
6      for op in list(block.ops):
7          name = op.name

```

```

8
9     # --- constants ---
10    if name == "arith.constant":
11        val = _const_value(op)
12        if val is None:
13            raise RuntimeError(f"Couldn't read constant value from
14                               op: {op}")
15        self._set(op.results[0], val)
16        continue
17
18    # --- high-level HC ops ---
19    if name == "hc.add":
20        a, b = op.operands
21        self._set(op.results[0], self._get(a) + self._get(b))
22        continue
23
24    if name == "hc.sub":
25        a, b = op.operands
26        self._set(op.results[0], self._get(a) - self._get(b))
27        continue
28
29    if name == "hc.mul":
30        a, b = op.operands
31        self._set(op.results[0], self._get(a) * self._get(b))
32        continue
33
34    if name == "hc.relu":
35        (x,) = op.operands
36        self._set(op.results[0], max(self._get(x), 0))
37        continue
38
39    # --- lowered arith ops (so you can run after lowering too) ---
40    if name == "arith.addi":
41        a, b = op.operands
42        self._set(op.results[0], self._get(a) + self._get(b))
43        continue
44
45    if name == "arith.subi":
46        a, b = op.operands
47        self._set(op.results[0], self._get(a) - self._get(b))
48        continue
49
50    if name == "arith.muli":
51        a, b = op.operands
52        self._set(op.results[0], self._get(a) * self._get(b))
53        continue

```

```

54         if name == "arith.maxsi":
55             a, b = op.operands
56             self._set(op.results[0], max(self._get(a), self._get(b)))
57             continue
58
59         # --- return ---
60         if name == "func.return":
61             if len(op.operands) != 1:
62                 raise RuntimeError(f"Expected func.return with 1 operand
63                                     , got: {op}")
64             return self._get(op.operands[0])
65
66         raise RuntimeError(f"Unsupported op in interpreter: {name}\nOp:
67                             {op}")
68
69         raise RuntimeError("Block ended without func.return")

```

Listing 63: run_block method implementation

This method executes a single basic block in a straight-line manner, meaning it processes operations sequentially without handling control flow such as branches or loops. It iterates through each operation in the block and dispatches behavior based on the operation's name.

For constants (`arith.constant`), it extracts the literal value and stores it in the interpreter's internal environment. The interpreter maintains a mapping from SSA values to concrete integer values.

For high-level operations (`hc.add`, `hc.sub`, `hc.mul`, `hc.relu`), it retrieves operand values, performs the corresponding Python arithmetic, and stores the computed result back into the environment. This directly implements the semantics of our high-level dialect.

It also supports lowered arithmetic operations (`arith.addi`, `arith.subi`, `arith.muli`, `arith.maxsi`), which allows the same interpreter to run code after lowering passes. This makes the interpreter useful both before and after middle-end transformations, reinforcing its role as a validation tool.

Next, implement `run_module` that calls `run_block`:

```

1 def run_module(self, module, func_name: str = "my_func") -> int:
2     """
3     Find func.func @func_name and execute its first block.
4     """
5     # module.body.blocks[0].ops usually contains top-level ops
6     for top_block in module.body.blocks:
7         for op in list(top_block.ops):
8             if isinstance(op, func.FuncOp):
9                 # Try to read the symbol name
10                sym = getattr(op, "sym_name", None)
11                name = None
12                if sym is not None:

```

```

13         name = _int_from_attr(getattr(sym, "data", None)) #
14             probably not int, ignore
15         # better: sym.data is usually a string
16         if hasattr(sym, "data"):
17             name = sym.data
18         else:
19             name = str(sym)
20
21         if name == func_name or (name is None and func_name == "
22             my_func"):
23             # assume single-block body for now
24             body_block = list(op.body.blocks)[0]
25             return self.run_block(body_block)
26
27         raise RuntimeError(f"Function not found: {func_name}")

```

Listing 64: run_module method implementation

This method searches the given module for a function with the specified name and executes it using the interpreter. It iterates over all top-level blocks in the module and then over their operations, looking specifically for instances of `func.FuncOp`, which represent function definitions in the IR.

For each function operation found, it attempts to extract the function's symbolic name (`sym_name`). Since attributes in xDSL are often wrapped objects, the code carefully accesses `sym.data` (which typically holds the actual string name) and falls back to string conversion if necessary.

Once the function name matches `func_name` (defaulting to `my_func`), the method assumes the function body contains a single block, retrieves that first block, and passes it to `self.run_block()` for execution.

To use it in our `hc_main.py`:

```

1 from simulator.interpreter import Interpreter
2 ...
3 def main():
4     # <Building module>
5     ...
6
7     hc_interpreter = Interpreter()
8     result_before = hc_interpreter.run_module(module, "my_func")
9     print("\nResult (interpreted, before lowering):", result_before)
10
11     # <Middle-end lowering part>
12     ...
13
14     lower_interpreter = Interpreter()
15     result_after = lower_interpreter.run_module(module, "my_func")
16     print("\nResult (interpreted, after lowering):", result_after)

```

18 Scf.for

In MLIR (and similarly in xDSL), SCF stands for Structured Control Flow, a dialect designed to represent high-level control flow constructs such as loops and conditionals in a structured and analyzable form. Unlike low-level branch-based control flow, SCF operations explicitly model structured constructs like if and for, making transformations and optimizations easier to reason about.

The scf.for operation represents a counted loop with a clear start value (lower bound), end value (upper bound), and step. It also supports loop-carried variables, allowing values to be passed from one iteration to the next in a well-defined SSA form. This makes scf.for particularly useful in ML compilers, where loops over tensor dimensions and iteration spaces are common and need to remain analyzable during optimization and lowering.

We will implement a pow operation in our high-level dialect and then lower it into an scf.for loop to express exponentiation using structured iteration. Extend the HC dialect:

```

1 @irdl_op_definition
2 class HCPow(IRDLOperation):
3     name = "hc.pow"
4     base = operand_def(IntegerType)
5     exp  = operand_def(IntegerType)
6     res  = result_def(IntegerType)
7
8     def verify_(self):
9         if self.base.type != self.exp.type or self.res.type != self.base
10            .type:
11             raise ValueError("hc.pow: base, exp, and res must have same
12                               integer type")

```

Listing 66: Pow operation definition

and register it:

```

1 (HCAdd, HCMul, HCSUB, HCRelu, HCPow),      # register operations
   here

```

Listing 67: Registering HCPow

In hc_lowering.py, provide additional imports for builtin, scf, Block and Region.

```

1 from xdsl.dialects import arith, builtin, scf
2 from xdsl.ir import Block, Operation, Region

```

Listing 68: Imports for lowering

Extend the list of supported operations:

```

1 if op.name not in ("hc.add", "hc.mul", "hc.sub", "hc.relu", "hc.pow"):
2     return

```

Listing 69: Supported operations

and provide lowering logic:

```

1 # ----- hc.pow -----
2 if op.name == "hc.pow":
3     base, exp = op.operands
4     ty = op.results[0].type # integer type
5
6     # constants for loop bounds/step
7     c0 = arith.ConstantOp.from_int_and_width(0, 32)
8     c1 = arith.ConstantOp.from_int_and_width(1, 32)
9
10    # init accumulator = 1
11    init = arith.ConstantOp.from_int_and_width(1, 32)
12
13    # scf.for (%i = 0; %i < exp; %i += 1) iter_args(%acc = init) -> (ty)
14    { %acc2 = muli %acc, base; scf.yield %acc2 }
15    body = Block(arg_types=[ty, ty])
16
17    # Better: use index for iv and keep acc as ty.
18    # Let's construct it robustly:
19    iv_ty = builtin.IndexType()
20    body = Block(arg_types=[iv_ty, ty])
21    iv, acc = body.args
22
23    mul = arith.MuliOp(acc, base)
24    body.add_ops([mul, scf.YieldOp(mul.result)])
25
26    loop = scf.ForOp(
27        lb=c0.result,          # lower bound
28        ub=exp,                # upper bound (dynamic)
29        step=c1.result,        # step
30        iter_args=[init.result],
31        body=Region(body),
32    )
33
34    # scf.ForOp returns the iter_args results
35    rewriter.replace_op(
36        op,
37        new_ops=[c0, c1, init, loop],
38        new_results=[loop.results[0]],
39        safe_erase=True,
40    )
41    return

```

Listing 70: hc.pow lowering

Try it in our hc_main.py:

```
1 from hc_dialect import HiCompiler, HCAdd, HCSub, HCMul, HCRelu, HCPow
2 ...
3 # Operand must be SSA value
4 c5 = arith.ConstantOp.from_int_and_width(3, 32)
5 hc_pow = HCPow(
6     operands=[c2, c4], # Base and exponent
7     result_types=[i32],
8 )
9 # Return the result
10 ret = func.ReturnOp(hc_pow.results[0])
11
12 # Block containing the constants + hc ops + return
13 block = Block(ops=[c1, c2, c3, c4, c5, hc_add, hc_mul, hc_sub, hc_relu,
    hc_pow, ret])
```

Lowered function should look something like (notice scf.for region-based operation):

```
1 builtin.module {
2   func.func @my_func() -> i32 {
3     ...
4     %10 = arith.constant 0 : i32
5     %11 = arith.constant 1 : i32
6     %12 = arith.constant 1 : i32
7     %13 = scf.for %14 = %10 to %2 step %11 iter_args(%15 = %12) -> (i32)
8     {
9       %16 = arith.muli %15, %2 : i32
10      scf.yield %16 : i32
11    }
12    func.return %13 : i32
13  }
```

Listing 71: Lowered example

To support pow operation in the simulator, we need to extend interpreter.py:

```
1 from xdsl.ir import BlockArgument, Operation, SSAValue
2 ...
3     if name == "hc.pow":
4         a, b = op.operands
5         self._set(op.results[0], self._get(a) ** self._get(b))
6         continue
7 ...
8     if name == "scf.for":
9         # Try: recognize "pow lowering" pattern and shortcut it.
```

```

10         if self._try_eval_pow_lowering(op):
11             continue
12         raise RuntimeError("Unsupported scf.for (not recognized
                               as pow lowering)")

```

Listing 72: Extending interpreter with pow operation

And provide helper function used for pow pattern evaluation:

```

1  def _try_eval_pow_lowering(self, op) -> bool:
2      """
3      Recognize and evaluate the specific lowering:
4      %res = scf.for %iv = %lb to %ub step %step iter_args(%acc = %
5          init) -> (i32) {
6          %m = arith.muli %acc, %base : i32
7          scf.yield %m : i32
8          }
9      If matched: set scf.for result(s) and return True, else False.
10     """
11     # Must have exactly 1 iter_arg result
12     if len(op.results) != 1:
13         return False
14
15     # Must have lb, ub, step at least
16     if len(op.operands) < 4:
17         return False
18
19     lb = self._get(op.operands[0])
20     ub = self._get(op.operands[1])
21     step = self._get(op.operands[2])
22     init = self._get(op.operands[3]) # first iter_arg init
23
24     # Typical pow lowering: lb=0, step=1, ub>=0
25     if lb != 0 or step != 1 or ub < 0:
26         return False
27
28     # Body: single block
29     try:
30         body_block = op.regions[0].blocks[0]
31     except Exception:
32         return False
33
34     ops = list(body_block.ops)
35     # Expect exactly: [arith.muli, scf.yield]
36     if len(ops) != 2:
37         return False
38
39     mul, yld = ops
40     if mul.name != "arith.muli" or yld.name != "scf.yield":

```

```

40         return False
41
42     # scf.yield must yield the mul result
43     if len(yld.operands) != 1 or yld.operands[0] is not mul.results
44       [0]:
45         return False
46
47     # mul operands should be: (iter_arg_block_arg, base_value)
48     # We don't want to fully interpret block args; just identify the
49     # "base" operand:
50     if len(mul.operands) != 2:
51         return False
52
53     a, b = mul.operands
54
55     # In xDSL, block args are typically instances of BlockArgument.
56     if isinstance(a, BlockArgument) and not isinstance(b,
57       BlockArgument):
58         base_val = self._get(b)
59     elif isinstance(b, BlockArgument) and not isinstance(a,
60       BlockArgument):
61         base_val = self._get(a)
62     else:
63         # Either both are block args or both are not -> not the
64         # simple pow pattern
65         return False
66
67     # Evaluate
68     result = init * (base_val ** ub)
69
70     self._set(op.results[0], result)
71     return True

```

Listing 73: Pow lowering recognition

19 Scf.if

`scf.if` is a structured control-flow operation used to express conditional execution in MLIR/xDSL. It evaluates a boolean condition and executes either the then region or the else region, optionally returning values from the chosen branch.

Using this mechanism, we can implement a max operation by comparing two values and selecting the larger one: if the first value is greater than the second, the then branch returns the first operand, otherwise the else branch returns the second.

First, extend dialect:

```

1 @irdl_op_definition

```

```

2 class HCTMax(IRDLOperation):
3     name = "hc.max"
4     lhs = operand_def(IntegerType)
5     rhs = operand_def(IntegerType)
6     res = result_def(IntegerType)
7
8     def verify_(self):
9         if self.lhs.type != self.rhs.type or self.res.type != self.lhs.
            type:
10             raise ValueError("hc.max: operands and result must have the
                same integer type")
11 ...
12 (HCTAdd, HCTMul, HCTSub, HCTRelu, HCTPow, HCTMax),      # register
            operations here

```

Listing 74: Adding max to hc_dialect.py

Implement lowering:

```

1 if op.name not in (
2     "hc.add", "hc.mul", "hc.sub", "hc.relu", "hc.pow", "hc.max"
3 ):
4     return
5 ...
6 # ----- hc.max -----
7 if op.name == "hc.max":
8     lhs, rhs = op.operands
9     ty = op.results[0].type
10
11     cmp = arith.CmpiOp(lhs, rhs, "sgt") # signed greater-than
12
13     then_block = Block(arg_types=[])
14     then_block.add_ops([scf.YieldOp(lhs)])
15
16     else_block = Block(arg_types=[])
17     else_block.add_ops([scf.YieldOp(rhs)])
18
19     if_op = scf.IfOp(
20         cmp.result,
21         [ty],
22         Region(then_block),
23         Region(else_block),
24     )
25
26     rewriter.replace_op(
27         op,
28         new_ops=[cmp, if_op],
29         new_results=[if_op.results[0]],
30         safe_erase=True,

```

```

31     )
32     return

```

Listing 75: HCMax lowering

Extend the interpreter with simple implementation, that just recognizes if pattern. Since lowered Max operation contains arith.cmpi, we have to address that in our interpreter, as well:

```

1  if name == "hc.max":
2      a, b = op.operands
3      self._set(op.results[0], max(self._get(a), self._get(b)))
4      continue
5  ...
6  if name == "arith.cmpi":
7      a, b = op.operands
8      lhs = self._get(a)
9      rhs = self._get(b)
10
11     # In xDSL: sgt == 4
12     pred = op.predicate.value.data
13
14     if pred != 4:
15         raise RuntimeError(f"Only sgt supported in interpreter, got
16             predicate: {pred}\nOp: {op}")
17
18     # signed greater-than
19     result = 1 if lhs > rhs else 0
20
21     self._set(op.results[0], result)
22     continue
23 ...
24 if name == "scf.if":
25     # Treat non-zero as True
26     cond = self._get(op.operands[0]) != 0
27
28     then_region = op.regions[0]
29     else_region = op.regions[1] if len(op.regions) > 1 else None
30
31     if cond:
32         yielded = self._eval_scf_if_region_yield(then_region)
33     else:
34         if else_region is None:
35             raise RuntimeError("scf.if has no else region but condition
36                 is false")
37         yielded = self._eval_scf_if_region_yield(else_region)
38
39     # Assume scf.if returns exactly one value
40     self._set(op.results[0], yielded)
41     continue

```

```

40 ...
41 # Helper method
42 def _eval_scf_if_region_yield(self, region) -> int:
43     """
44     Simplified: Assume region has 1 block and the last op is scf.yield
45     with 1 operand.
46     We ignore everything else in the region and just read the yielded
47     SSA value.
48     """
49     block = list(region.blocks)[0]
50     last = list(block.ops)[-1]
51     if last.name != "scf.yield" or len(last.operands) != 1:
52         raise RuntimeError(f"Expected region to end with scf.yield(1),
53                             got: {last}")
54     return self._get(last.operands[0])

```

Listing 76: Interpreting operation

Use newly created HCMax in our `hc_main.py` and try different values.

As an exercise, extend dialect with HCMin operation (hint: use `slg` for signed less-than operation). To be able to interpret new operation, you will need to adjust `arith.cmpi` section in `interpreter.py`:

```

1 if name == "arith.cmpi":
2     a, b = op.operands
3     lhs = self._get(a)
4     rhs = self._get(b)
5
6     pred = op.predicate.value.data
7
8     # In xDSL: sgt == 4; lgt == 2
9     if pred == 4: # signed greater-than
10         result = 1 if lhs > rhs else 0
11     elif pred == 2: # signed less-than
12         result = 1 if lhs < rhs else 0
13     else:
14         raise RuntimeError(
15             f"Only sgt and slt supported in interpreter, got predicate:
16             {pred}\nOp: {op}"
17         )
18     self._set(op.results[0], result)
19     continue

```

Listing 77: Extending arith.cmpi interpretation

20 Front end

So far, we have been constructing modules manually using our HC dialect operations. In this step, we will move toward a more realistic workflow by loading a model from an ONNX file instead of building the IR by hand. Since we do not yet have an ONNX model prepared, we will first create a small example model and export it to ONNX format. Then we will implement a simple front end that reads the ONNX graph and maps its nodes to our corresponding HC operations, producing an HC module that can be further compiled.

To organize this part of the project, we will create a folder with the following structure:

```
1 front_end
2 |-- __init__.py
3 |-- build_model.py
4 |-- loader.py
```

Listing 78: Front end folder structure

In `build_model.py`, import `onnx` related packages and define final model path:

```
1 import onnx
2 from onnx import helper, TensorProto
3
4 MODEL_PATH = "score_model.onnx"
```

Listing 79: onnx imports

```
1 def build_score_model(path: str = MODEL_PATH):
2     # -----
3     # Parameters
4     # -----
5     MU = 40.0
6     ALPHA = 2.0
7     P = 2.0
8     LO = 0.0
9     HI = 1000.0
10
11     # -----
12     # Input / Output
13     # -----
14     x = helper.make_tensor_value_info("x", TensorProto.FLOAT, [])
15     y = helper.make_tensor_value_info("score", TensorProto.FLOAT, [])
16
17     # -----
18     # Constant initializers
19     # -----
20     mu_init = helper.make_tensor("mu", TensorProto.FLOAT, [], [MU])
21     a_init = helper.make_tensor("alpha", TensorProto.FLOAT, [], [ALPHA])
22     p_init = helper.make_tensor("p", TensorProto.FLOAT, [], [P])
```

```

23     lo_init = helper.make_tensor("lo", TensorProto.FLOAT, [], [LO])
24     hi_init = helper.make_tensor("hi", TensorProto.FLOAT, [], [HI])
25
26     # -----
27     # Graph nodes
28     # -----
29     nodes = [
30         # d1 = x - mu
31         helper.make_node("Sub", ["x", "mu"], ["d1"], name="sub_x_mu"),
32
33         # d2 = mu - x
34         helper.make_node("Sub", ["mu", "x"], ["d2"], name="sub_mu_x"),
35
36         # abs_d = max(d1, d2)
37         helper.make_node("Max", ["d1", "d2"], ["abs_d"], name="abs_max")
38         ,
39
40         # e = pow(abs_d, p)
41         helper.make_node("Pow", ["abs_d", "p"], ["e"], name="pow"),
42
43         # s = alpha * e
44         helper.make_node("Mul", ["alpha", "e"], ["s"], name="scale"),
45
46         # lower = max(s, lo)
47         helper.make_node("Max", ["s", "lo"], ["lower"], name="clamp_low"
48             ),
49
50         # clamped = min(lower, hi)
51         helper.make_node("Min", ["lower", "hi"], ["clamped"], name="
52             clamp_high"),
53
54         # score = relu(clamped)
55         helper.make_node("Relu", ["clamped"], ["score"], name="relu"),
56     ]
57
58     graph = helper.make_graph(
59         nodes=nodes,
60         name="ScoreGraph",
61         inputs=[x],
62         outputs=[y],
63         initializer=[mu_init, a_init, p_init, lo_init, hi_init],
64     )
65
66     model = helper.make_model(
67         graph,
68         opset_imports=[helper.make_opsetid("", 13)],
69         producer_name="score_builder",

```

```

67     )
68
69     onnx.checker.check_model(model)
70     onnx.save(model, path)
71     print(f"Saved ONNX model to: {path}")
72
73
74 if __name__ == "__main__":
75     build_score_model()

```

Listing 80: Building model

Run the script and observe generated model using `netron.app` website. Then, in `loader.py`, we will build HC module based on provided onnx model. Start with importing required packages and our dialect:

```

1 import onnx
2 from xdsl.dialects import func, arith
3
4 from xdsl.context import Context
5 from xdsl.dialects.builtin import ModuleOp, i32
6 from xdsl.ir import Region, Block
7
8 from hc_dialect import HiCompiler, HCAdd, HCSUB, HCMul, HCRelu, HCPow,
   HCMax, HCMin

```

Listing 81: Loader imports

Define helper functions for types conversion:

```

1 # -----
2 # Helper functions
3 # -----
4 def _onnx_tensor_to_scalar_int(tensor_proto) -> int:
5     arr = onnx.numpy_helper.to_array(tensor_proto)
6     # Restriction: scalar or 1-element tensor
7     if arr.size != 1:
8         raise ValueError(f"Only scalar/1-element constants supported,
9                             got shape {arr.shape}")
10    return int(arr.reshape(()))
11
12 def _make_i32_const(value: int) -> arith.ConstantOp:
13     return arith.ConstantOp.from_int_and_width(int(value), 32)

```

Listing 82: Defining loader helper functions

```

1 # -----
2 # Loader
3 # -----
4 def import_onnx_to_hc_module(

```

```

5     ctx: Context, onnx_path: str, fn_name: str="main"
6 ) -> ModuleOp:
7     model = onnx.load(onnx_path)
8     graph = model.graph
9
10    # Map ONNX value names -> SSAValue (results)
11    env: dict[str, object] = {}
12
13    # Assume one scalar input
14    # Map ONNX graph input name to function/block argument
15    entry_block = Block(arg_types=[i32])
16    x_name = graph.input[0].name
17    env[x_name] = entry_block.args[0]
18
19    ops = []
20
21    # 1) Initializers (weights/constants stored in graph.initializer)
22    for init in graph.initializer:
23        v = _onnx_tensor_to_scalar_int(init)
24        c = _make_i32_const(v)
25        ops.append(c)
26        env[init.name] = c.result
27
28    # 2) Constant nodes (op_type == "Constant")
29    # ONNX Constant usually stores a tensor in attribute "value"
30    def emit_constant_node(node):
31        value_attr = None
32        for a in node.attribute:
33            if a.name == "value":
34                value_attr = a.t
35        if value_attr is None:
36            raise ValueError("Constant node without 'value' attribute")
37        v = _onnx_tensor_to_scalar_int(value_attr)
38        c = _make_i32_const(v)
39        ops.append(c)
40        env[node.output[0]] = c.result
41
42    # 3) Node lowering
43    for node in graph.node:
44        if node.op_type == "Constant":
45            emit_constant_node(node)
46            continue
47
48    def get(name: str):
49        if name not in env:
50            raise KeyError(f"ONNX value not found yet: {name} (node
                                {node.op_type})")

```

```

51         return env[name]
52
53     if node.op_type == "Add":
54         a = get(node.input[0])
55         b = get(node.input[1])
56         hc_add = HCAdd(operands=[a, b], result_types=[i32])
57         ops.append(hc_add)
58         env[node.output[0]] = hc_add.results[0]
59         continue
60
61     if node.op_type == "Sub":
62         a = get(node.input[0])
63         b = get(node.input[1])
64         hc_sub = HCSUB(operands=[a, b], result_types=[i32])
65         ops.append(hc_sub)
66         env[node.output[0]] = hc_sub.results[0]
67         continue
68
69     if node.op_type == "Mul":
70         a = get(node.input[0])
71         b = get(node.input[1])
72         hc_mul = HCMul(operands=[a, b], result_types=[i32])
73         ops.append(hc_mul)
74         env[node.output[0]] = hc_mul.results[0]
75         continue
76
77     if node.op_type == "Relu":
78         x = get(node.input[0])
79         hc_relu = HCRelu(operands=[x], result_types=[i32])
80         ops.append(hc_relu)
81         env[node.output[0]] = hc_relu.results[0]
82         continue
83
84     if node.op_type == "Pow":
85         a = get(node.input[0])
86         b = get(node.input[1])
87         hc_pow = HCPow(operands=[a, b], result_types=[i32])
88         ops.append(hc_pow)
89         env[node.output[0]] = hc_pow.results[0]
90         continue
91
92     if node.op_type == "Max":
93         a = get(node.input[0])
94         b = get(node.input[1])
95         hc_max = HCMax(operands=[a, b], result_types=[i32])
96         ops.append(hc_max)
97         env[node.output[0]] = hc_max.results[0]

```

```

98         continue
99
100     if node.op_type == "Min":
101         a = get(node.input[0])
102         b = get(node.input[1])
103         hc_min = HCMin(operands=[a, b], result_types=[i32])
104         ops.append(hc_min)
105         env[node.output[0]] = hc_min.results[0]
106         continue
107
108     raise NotImplementedError(f"Unsupported ONNX op: {node.op_type}")
109
110 # 4) Determine outputs (graph.output)
111 if len(graph.output) != 1:
112     raise ValueError("For now: expect exactly 1 model output")
113 out_name = graph.output[0].name
114 if out_name not in env:
115     raise KeyError(f"Graph output {out_name} not produced")
116 ret = func.ReturnOp(env[out_name])
117 ops.append(ret)
118
119 for op in ops:
120     entry_block.add_op(op)
121
122 fn = func.FuncOp(fn_name, ([i32], [i32]))
123 fn.body = Region(entry_block)
124
125 return ModuleOp(ops=[fn])

```

Listing 83: Defining model loader

This function converts a neural-network model stored in the ONNX format into an intermediate representation (IR) expressed using the HC dialect in xDSL. The goal is to transform the graph-based ONNX model into a sequence of SSA operations that can be further processed, optimized, or executed by your compiler infrastructure.

The loader reads the ONNX graph and builds an internal mapping between ONNX value names and SSA values in the IR. Model inputs are represented as function arguments, while constants and parameters are turned into constant operations. Each ONNX node is then translated into an equivalent operation in the HC dialect, gradually constructing the computation inside the function's entry block.

Once all nodes are translated, the loader determines the graph's output and adds a return operation to produce the final value. The resulting operations are placed into a function, and that function is wrapped inside a module, producing a complete IR representation of the original ONNX model that can be executed or transformed later in the compilation pipeline.

Since the module we just build has one argument that we can pass as an input, we need to

modify our `interpreter.py` to also support this feature by changing `run\_module` signature and providing logic for args handling:

```

1 def run_module(self, module,  args: list[int], func_name: str = "my_func
  ") -> int:
2   ...
3       if name == func_name or (name is None and func_name
  == "my_func"):
4           # assume single-block body for now
5           body_block = list(op.body.blocks)[0]
6
7           if len(args) != len(body_block.args):
8               raise RuntimeError(
9                   f"Expected {len(body_block.args)} args,
10                      got {len(args)}"
11               )
12           for block_arg, value in zip(body_block.args,
13               args):
14               self._set(block_arg, value)
15           return self.run_block(body_block)

```

Listing 84: Argument handling in interpreter

Now, in our `hc_main.py`, we can entirely remove `build_module` function and use loader.

```

1 from front_end.build_model import MODEL_PATH
2 from front_end.loader import import_onnx_to_hc_module

```

Listing 85: `hc_main.py` imports

Load the model and run it through interpreter.

```

1 def main():
2     ctx = build_context()
3     module = import_onnx_to_hc_module(ctx, MODEL_PATH, "my_func")
4
5     print("=== HIGH LEVEL (hc.*) ===")
6     print(module)
7     input_val = [31]
8
9     hc_interpreter = Interpreter()
10    result_before = hc_interpreter.run_module(module, args=input_val,
11        func_name="my_func")
12    print("\nResult (interpreted, before lowering):", result_before)
13
14    config = MiddleEndPipelineConfig(
15        apply_lowering=True,
16        apply_constant_folding=False,
17        apply_dce=False,
18        run_analysis=False,

```

```

18         debug_mode=False,
19     )
20
21     middle_end_pipeline = MiddleEndPipeline(config)
22     middle_end_pipeline.apply_passes(module)
23     print("=== After lowering ===")
24     print(module)
25
26     lower_interpreter = Interpreter()
27     result_after = lower_interpreter.run_module(module, args=input_val,
28         func_name="my_func")
29     print("\nResult (interpreted, after lowering):", result_after)

```

Listing 86: Using loader in main

Try different values of `input_val`, and observe how output changes when we diverge from the value 40.

21 Batch

A batch refers to processing multiple inputs together instead of executing a computation separately for each input. In practice, it means running the same function or model multiple times with different input values while treating them as a group.

Batch execution is useful because it allows repeated computations to be organized and handled more efficiently. It simplifies workflows where the same model must be evaluated on many inputs, such as testing, data processing, or machine learning inference.

Using batches is also convenient because it reduces repetitive code and provides a clearer interface for running multiple evaluations, while often enabling better performance in real systems that can process many inputs in parallel.

At the moment, we have a `run_module` that executes a single function from the module using the interpreter. It binds the provided runtime values to the function's input arguments (entry block arguments) and then interprets the operations in the block sequentially until a `func.return` is reached, producing the final result.

`run_module_batch` extends this idea to batch execution. Instead of running the function once, it repeatedly calls `run_module` with different argument sets, allowing the same function to be evaluated multiple times with different inputs and collecting all results into a list.

Start implementation by extending `Interpreter` class with `run_module_batch` method:

```

1     def run_module_batch(
2         self,
3         module,
4         batch_args: list[list[int]],
5         func_name: str = "my_func"
6     ) -> list[int]:
7         results = []
8

```

```

9         for args in batch_args:
10             # reset environment for each independent execution
11             self.env = {}
12             result = self.run_module(module, args=args, func_name=
13                                     func_name)
14             results.append(result)
15
16     return results

```

Listing 87: Batch execution implementation

Then we can modify our `hc_main.py` to use batch execution. Add a couple of input batches and replace `hc_interpreter.run_module` with `hc_interpreter.run_module_batch` calls:

```

1     input_batches = [[65], [44], [40], [35], [30], [18]]
2     ...
3     result_before = hc_interpreter.run_module_batch(
4         module, batch_args=input_batches, func_name="my_func"
5     )
6     ...
7     result_after = lower_interpreter.run_module_batch(
8         module, batch_args=input_batches, func_name="my_func"
9     )

```

Listing 88: Batch execution

You should see score results spanning the range from the LO (0) up to the HI parameter value (1000) defined in `build_model.py`.

22 Back end

As a follow-up, we will create Bash scripts that automate the compilation pipeline for our model. These scripts will take the MLIR program produced after lowering and progressively convert it into LLVM IR, and then into a final executable. By using LLVM tools in the scripts, we can easily compile and run our MLIR-based model from the command line.

```

1 full_compiler.sh
2 back_end
3 |-- back_end.sh

```

Listing 89: New scripts for the pipeline

In the `back_end.sh` script, implement the first step of progressive compilation.

```

1 #/bin/sh
2 MODEL_NAME="$1"
3 MODEL_STEM="${MODEL_NAME%.onnx}"
4 echo "Compiling backend..."
5
6 # More robust way to determine script's location

```

```

7 SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
8 TOOLCHAIN_BIN_DIR="$SCRIPT_DIR/../../LLVM_IR/llvm-project/build/bin"
9 BUILD_DIR=${SCRIPT_DIR}/../build
10
11 ${TOOLCHAIN_BIN_DIR}/mlir-opt ${BUILD_DIR}/${MODEL_STEM}_lowered.mlir \
12     -convert-scf-to-cf \
13     -convert-cf-to-llvm \
14     -convert-arith-to-llvm \
15     -convert-func-to-llvm \
16     -reconcile-unrealized-casts \
17     -o ${BUILD_DIR}/${MODEL_STEM}_llvm.mlir
18 echo "Created file: ${BUILD_DIR}/${MODEL_STEM}_llvm.mlir"

```

Listing 90: Creating LLVM IR by using mlir-opt

This command runs mlir-opt to apply a sequence of lowering passes that convert the MLIR program into the LLVM dialect. The passes progressively transform structured control flow (scf), control flow (cf), arithmetic operations (arith), and functions (func) into their LLVM equivalents. The result is written to a new file containing LLVM-level MLIR, which can later be translated into LLVM IR and compiled into an executable. Use it from our main pipeline script, together with hc_main.py. Define full_compiler.py script with following content:

```

1 #!/usr/bin/env bash
2
3 MODEL_NAME=${1:-score_model.onnx}
4
5 echo "Using model: $MODEL_NAME"
6
7 python3 hc_main.py "$MODEL_NAME"
8 back_end/back_end.sh "$MODEL_NAME"

```

Listing 91: Full compiler script implementation

As you can see, for back end, we are using results previously generated in hc_main.py step, so we need to adjust that script to produce wanted files. In hc_main.py, add imports:

```

1 from hc_dialect import HiCompiler
2 from front_end.build_model import build_score_model, MODEL_PATH

```

Listing 92: Needed imports

Our hc_main.py now accepts model path argument, that we use to import onnx model.

```

1 def main():
2     # Optional argument
3     model_arg = sys.argv[1] if len(sys.argv) > 1 else MODEL_PATH
4     model_path = Path(model_arg)
5
6     # Extract base name (without extension)
7     model_name = model_path.stem

```

```

8
9     build_dir = Path("build")
10    build_dir.mkdir(exist_ok=True)
11
12    if not model_path.exists():
13        candidate = build_dir / model_path.name
14        if candidate.exists():
15            model_path = candidate
16        else:
17            model_path = build_dir / model_path.name
18            print(f"Model not found, building: {model_path}")
19            build_score_model(str(model_path))
20
21    original_file = build_dir / f"{model_name}_original.mlir"
22    lowered_file = build_dir / f"{model_name}_lowered.mlir"
23
24    ctx = build_context()
25    module = import_onnx_to_hc_module(
26        ctx, onnx_path=str(model_path), fn_name="my_func"
27    )
28
29    print("Saving high-level module...")
30    with open(original_file, "w", encoding="utf-8") as f:
31        f.write(str(module))
32
33    config = MiddleEndPipelineConfig(
34        apply_lowering=True,
35        apply_constant_folding=False,
36        apply_dce=False,
37        run_analysis=False,
38        debug_mode=False,
39    )
40
41    middle_end_pipeline = MiddleEndPipeline(config)
42    middle_end_pipeline.apply_passes(module)
43
44    print("Saving lowered module...")
45    with open(lowered_file, "w", encoding="utf-8") as f:
46        f.write(str(module))

```

Listing 93: Modified main function

Remove unnecessary imports from `hc_main.py`. All generated files will be placed inside `build` folder.

You need to give execution rights to both newly created bash scripts:

```

1  chmod +x full_compiler.sh
2  chmod +x back_end/back_end.sh

```

Listing 94: Changing execution rights

If you run the `full_compiler.sh` script now, it will fail, since we need to fix a bug that we introduced when we implemented pow lowering. We need to use an `IndexType` for `scf.for` upper bound. Change lowering part of Pow operation in `hc_lowering` file:

```
1      # ----- hc.pow -----
2      if op.name == "hc.pow":
3          base, exp = op.operands
4          ty = op.results[0].type # integer type
5
6          iv_ty = builtin.IndexType()
7
8          # loop-control constants must be index
9          c0 = arith.ConstantOp.from_int_and_width(0, iv_ty)
10         c1 = arith.ConstantOp.from_int_and_width(1, iv_ty)
11
12         # exponent is i32 -> cast to index for scf.for upper bound
13         exp_idx = arith.IndexCastOp(exp, iv_ty)
14
15         # init accumulator = 1
16         init = arith.ConstantOp.from_int_and_width(1, 32)
17
18         # scf.for body arguments: (iv: index, acc: i32)
19         body = Block(arg_types=[iv_ty, ty])
20         iv, acc = body.args
21
22         mul = arith.MuliOp(acc, base)
23         body.add_ops([mul, scf.YieldOp(mul.result)])
24
25         loop = scf.ForOp(
26             lb=c0.result,          # lower bound
27             ub=exp_idx.result,     # upper bound
28             step=c1.result,        # step
29             iter_args=[init.result],
30             body=Region(body),
31         )
32
33         # scf.ForOp returns the iter_args results
34         rewriter.replace_op(
35             op,
36             new_ops=[c0, c1, exp_idx, init, loop],
37             new_results=[loop.results[0]],
38             safe_erase=True,
39         )
40     return
```

Listing 95: Final Pow lowering

Now, if we run the script, we should see something like:

```
1 $ ./full_compiler.sh model_score.onnx
2 Using model: model_score.onnx
3 Model not found, building: build/model_score.onnx
4 Saved ONNX model to: build/model_score.onnx
5 Saving high-level module...
6 Saving lowered module...
7 Compiling backend...
8 Created file: <some absolute path>/src/back_end/./build/
   model_score_llvm.mlir
```

Listing 96: Output of the compiler script

Continue building our back end side. Add this to your `back_end.sh` script:

```
1 ${TOOLCHAIN_BIN_DIR}/mlir-translate ${BUILD_DIR}/${MODEL_STEM}_llvm.mlir
   \
2   -mlir-to-llvmir -o ${BUILD_DIR}/${MODEL_STEM}_out.ll
3 echo "Created file: ${BUILD_DIR}/${MODEL_STEM}_out.ll"
```

Listing 97: Using mlir-translate

This command uses `mlir-translate` to convert the MLIR file in the LLVM dialect into LLVM IR.

- The LLVM dialect is a representation used inside MLIR that models LLVM-style operations while still staying within the MLIR infrastructure. It is designed to look very similar to LLVM instructions but remains compatible with MLIR features such as dialects, passes, and transformations. In other words, it is a bridge between MLIR and LLVM, allowing MLIR programs to be gradually lowered toward LLVM.
- LLVM IR (Intermediate Representation) is the core low-level language used by the LLVM compiler infrastructure. It is a platform-independent representation that is close to machine instructions but still abstract enough to allow further optimizations and analysis.

To produce assembly code, we can extend script with following command:

This command uses `llc`, the LLVM static compiler, to convert LLVM IR into assembly code for the target architecture.

```
1 ${TOOLCHAIN_BIN_DIR}/llc -O2 -filetype=asm \
2   ${BUILD_DIR}/${MODEL_STEM}_out.ll -o ${BUILD_DIR}/${MODEL_STEM}_out.
   s
3 echo "Created file: ${BUILD_DIR}/${MODEL_STEM}_out.s"
```

Listing 98: Using llc for assembly

The input file `${MODEL_STEM}_out.ll` contains LLVM IR, and the options `-O2` and `-filetype=asm` tell `llc` to apply level-2 optimizations and generate assembly output.

To generate an object file, add:

```
1 ${TOOLCHAIN_BIN_DIR}/llc -filetype=obj ${BUILD_DIR}/${MODEL_STEM}_out.ll
  \
2 -o ${BUILD_DIR}/${MODEL_STEM}_out.o
3 echo "Created file: ${BUILD_DIR}/${MODEL_STEM}_out.o"
```

Listing 99: Using `llc` for object file

Command runs `llc`, the LLVM static compiler, to translate the LLVM IR file into a target-specific object file. The option `-filetype=obj` instructs `llc` to generate an object file (`.o`), which can later be linked with other object files or libraries to produce the final executable.

Note: You can use `llvm-objdump -d build/model_score_out.o` to inspect object file. `llvm-objdump` is a part of LLVM toolchain that lets us verify that our `MLIR->LLVM->machinecode` pipeline produced the expected instructions.

23 llvm-project installation

Install dependencies:

```
1 sudo apt update
2 sudo apt install -y \
3     git cmake ninja-build \
4     clang lld \
5     build-essential \
6     python3 python3-pip \
7     libtinfo-dev zlib1g-dev
```

Listing 100: Dependencies installation

Clone `llvm-project`:

```
1 git clone https://github.com/llvm/llvm-project.git
2 cd llvm-project
```

Listing 101: Dependencies installation

Create a build folder inside of it:

```
1 mkdir build
2 cd build
```

Listing 102: Creating build directory

Configure with CMake and Ninja generator:

```
1 cmake -G Ninja ../llvm \
2 -DLLVM_ENABLE_PROJECTS="clang;mlir;lld" \
3 -DCMAKE_BUILD_TYPE=Release \
```

```
4 -DLLVM_ENABLE_ASSERTIONS=ON
```

Listing 103: Configuring with CMake

Build everything:

```
1 ninja -j 2
```

Listing 104: Building repository

Verify tools:

```
1 mlir-opt --version
2 mlir-translate --version
3 llc --version
4 clang --version
```

Listing 105: Tools verification

24 Harness

We can also create a small driver program that invokes compiled code and allows it to be executed and tested inside a normal application. We implement the harness in C because the compiled function follows the C calling convention, making it straightforward to import it with `extern` and link it directly into the executable.

Create a file `back_end/harness.c`:

```
1 extern int my_func(int);
2
3 #include <stdio.h>
4 #include <stdlib.h>
5
6 int main(int argc, char **argv) {
7     int a = atoi(argv[1]); // convert command-line string to int
8     printf("Result of execution for (%d): %d\n", a, my_func(a));
9 }
```

Listing 106: Simple C application

And extend `back_end.sh`:

```
1 clang ${SCRIPT_DIR}/harness.c ${BUILD_DIR}/${MODEL_STEM}_out.o -o \
2     ${BUILD_DIR}/${MODEL_STEM}_run
3
4 echo "Created executable: ${BUILD_DIR}/${MODEL_STEM}_run"
5 echo "Running executable..."
6
7 ${BUILD_DIR}/${MODEL_STEM}_run 42
```

By running our function with different values, we can see that results will produce a U-shaped curve with a minimum at 40, like expected.

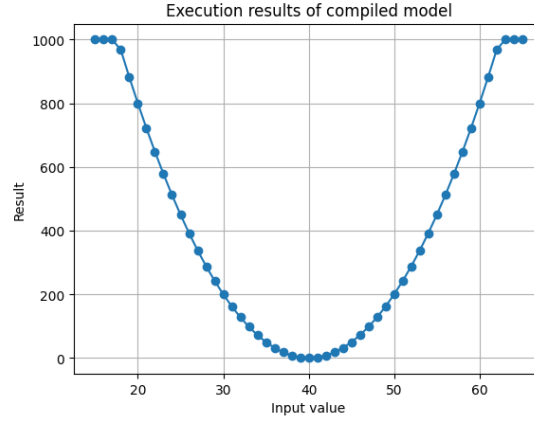


Figure 1: Execution results

25 Conclusion

This tutorial demonstrated the complete journey of compiling a simple machine learning computation from a custom dialect to an executable program. We began by defining a HiCompiler dialect, introducing domain-specific operations that represent high-level computations. Next, we implemented lowering rules that translate these operations into standard MLIR dialects such as arith, enabling the program to move closer to hardware-level representation. After lowering, we applied compiler optimizations like constant folding and dead code elimination to simplify the program and improve efficiency. Once the IR was normalized and optimized, we used MLIR passes to convert the program into the LLVM dialect, which bridges MLIR and the LLVM compiler infrastructure. The LLVM dialect was then translated into LLVM IR, a low-level, platform-independent intermediate representation.

Using LLVM tools, we compiled the LLVM IR into assembly and object files, preparing it for execution on the target machine. We then linked the compiled object with a small C harness, allowing the generated function to be called from a standard executable program. This final step enabled us to run and test the compiled model, confirming that the entire compilation pipeline works end to end. Overall, the tutorial illustrated how modern compiler infrastructures like MLIR and LLVM allow developers to design domain-specific languages, perform powerful optimizations, and ultimately generate efficient native executables.